

LAB 3 – INTELLIGENT DECISION-MAKING IN ROBOTICS
IGNACIO GONZÁLEZ MELÉNDEZ
NIA: 100522976

EXERCISE 1:

In this first exercise, we must infer the most likely sequence of hidden states for a given observation sequence using the function `predict()`. With this sequence we can know the most likely path to be followed by the robot among the possible hidden states.

Firstly, the sequence of observed outputs given in the laboratory statement [0, 1, 2, 3, 4, 5] is used. In Figure 1 we can see the hidden statements sequence regarding these observations.

```
SEQUENCE OF HIDDEN STATES  
[0 1 0 1 2 2]
```

Figure 1: Sequence of Hidden States I

Then, if we change the sequence of observations to [0, 1, 1, 3, 4, 2], for example, we get a different sequence of hidden states as can be seen in Figure 2. This is because in an HMM, `predict()` uses the Viterbi algorithm, which finds the most likely hidden-state path for the observation sequence. Changing any observation affects the global optimization, so the model recalculates the full path. This is why even early hidden states can change when later observations are modified.

```
SEQUENCE OF HIDDEN STATES  
[0 1 1 1 2 0]
```

Figure 2: Sequence of Hidden States II

EXERCISE 2:

In Figure 3 it can be seen the posterior probabilities, this is the probability of a state given the observation. Each row in the posterior probability table corresponds to one observation in the sequence. In this case, we can see that for observation 0 and 2 the most likely hidden state is Smooth, for observations 1 and 3 it is Rough and for the last 2 ones, observations 4 and 5, the most likely is Slippery.

```
POSTERIOR PROBABILITIES  
[[0.96599462 0.03400538 0.        ]  
 [0.36775461 0.63224539 0.        ]  
 [0.77842062 0.06437652 0.15720286]  
 [0.15399853 0.59837062 0.24763084]  
 [0.06975433 0.06062063 0.86962504]  
 [0.06201751 0.19204555 0.74593694]]
```

Figure 3: Posterior Probabilities Table

EXERCISE 3:

In Figure 4 we have the log probability of each sequence of observations. The log probability provides us with a way to measure the correct fitting of our observations sequence over the model. We can clearly see that as we are reducing the number of observations we get a better log probability, which is correct because it will be easier to determine the hidden state for our model.

```
-10.852225158676852  
-5.083631884064014  
-1.8078888511579385
```

Figure 4: Log Probability for each Observation Sequence

EXERCISE 4:

For this exercise, the main assumption that I made was the fact that Maggie is more likely to start the simulations alone and for the 10 iterations simulation it will always start alone, I decided this to simplify the simulation and to try to recreate a real scenario. The transition probabilities were selected according to the fact that it is more likely to stay in the same state than change to another one. However, the emission probabilities were selected following the idea that when Maggie is alone it is more likely to not detect anything, when it is in idle state it is more likely to detect at least something, and if it is interacting it is more likely to detect all features.

EXERCISE 5:

In Figure 5, we can see the hidden states regarding the simulated observations. In this case, as the first observation was detecting just the body, the most probable hidden state was idle. After this, the robot detected almost everything, which means that the state is going to be interacting. Also, as the transition probabilities establishes that it is more likely to stay in the same state, as said in the previous exercise, the most probable future state is interacting with observation number 7, which consists of detecting every part of the human.

```
Simulated Observations: [4 6 7 7 6]
Hidden States: ['idle', 'interacting', 'interacting', 'interacting', 'interacting']
Most probable future state: 2
Most probable future observation: 7
```

Figure 5: Maggie Simulation with Observations

EXERCISE 6:

For exercise 6, I did a simulation of 10 Maggie's iterations. Maggie will always start in alone state and will be changing its real state regarding the transition probabilities. Also, an observation is generated following the emission probabilities using the real state. Once an observation has been generated, the model will predict the state and print a message regarding that state. In Figure 6 this simulation can be seen.

```
Iteration 1
Observation: 0
State: alone
Maggie is sleeping

Iteration 2
Observation: 6
State: idle
Maggie calling User

Iteration 3
Observation: 6
State: idle
Maggie calling User

Iteration 4
Observation: 4
State: idle
Maggie calling User

Iteration 5
Observation: 4
State: idle
Maggie calling User

Iteration 6
Observation: 3
State: alone
Maggie is sleeping

Iteration 7
Observation: 6
State: idle
Maggie calling User

Iteration 8
Observation: 6
State: idle
Maggie calling User

Iteration 9
Observation: 6
State: idle
Maggie calling User

Iteration 10
Observation: 6
State: idle
Maggie calling User

=====SIMULATION COMPLETED=====
```

Figure 6: 10 Iterations Simulation of Maggie's Behavior

EXERCISE 7:

For completing this exercise, I used generative tools. Precisely, they were used for completing the Transition Model class functions. However, I had to make some changes in order to achieve the correct functionality of the robot.

EXERCISE 8:

In the given code, the robot is always starting in shelf number 1, and the object is established in shelf number 3. If the robot only has 3 steps, as given in the code, it won't be able to get to the object.

However, if we increment this number of steps to 10 the robot will be able to achieve the object by going to shelf number 2 and making an inspect action in the correct shelf. Also, the fact that we have noise makes the robot not completely discard a shelf if an inspection procedure does not give a proper result. In Figure 7, a 10-step shelf robot simulation can be seen. When the robot gets to the object, he will remain in that position, increasing the probability of finding the object on that shelf, reducing the effect of noise.

```
** Testing value iteration **
==== Step 1 ====
True state: shelf1
Belief: (ObjectState(shelf1): 0.25, ObjectState(shelf2): 0.25, ObjectState(shelf3): 0.25, ObjectState(shelf4): 0.25)
Action: inspect
Reward: -10.0
-> Observation: not_found
==== Step 2 ====
True state: shelf1
Belief: (ObjectState(shelf1): 0.3148148148148148, ObjectState(shelf2): 0.05555555555555556, ObjectState(shelf3): 0.3148148148148148, ObjectState(shelf4): 0.3148148148148148)
Action: move_left
Reward: -1.0
-> Observation: not_found
==== Step 3 ====
True state: shelf1
Belief: (ObjectState(shelf1): 0.3518518518518519, ObjectState(shelf2): 0.31703703703703706, ObjectState(shelf3): 0.3122222222222223, ObjectState(shelf4): 0.018888888888888896)
Action: inspect
Reward: -10.0
-> Observation: not_found
==== Step 4 ====
True state: shelf1
Belief: (ObjectState(shelf1): 0.41977522487102276, ObjectState(shelf2): 0.06866956905314435, ObjectState(shelf3): 0.3848890508429067, ObjectState(shelf4): 0.12666615523292613)
Action: move_right
Reward: -1.0
-> Observation: not_found
==== Step 5 ====
True state: shelf2
Belief: (ObjectState(shelf1): 0.021675456934082585, ObjectState(shelf2): 0.4053821368980265, ObjectState(shelf3): 0.08696496039584972, ObjectState(shelf4): 0.4859774457720412)
Action: inspect
Reward: 50.0
-> Observation: found
==== Step 6 ====
True state: shelf2
Belief: (ObjectState(shelf1): 0.03689840330729027, ObjectState(shelf2): 0.7560781352617959, ObjectState(shelf3): 0.05332295367734757, ObjectState(shelf4): 0.15370050775356636)
Action: inspect
Reward: 50.0
-> Observation: found
==== Step 7 ====
True state: shelf2
Belief: (ObjectState(shelf1): 0.028850163413976126, ObjectState(shelf2): 0.8897086285385498, ObjectState(shelf3): 0.031777004892936514, ObjectState(shelf4): 0.0496642031545375)
Action: inspect
Reward: 50.0
-> Observation: found
==== Step 8 ====
True state: shelf2
Belief: (ObjectState(shelf1): 0.024674032098611565, ObjectState(shelf2): 0.9221704117115594, ObjectState(shelf3): 0.02514343023896386, ObjectState(shelf4): 0.02801212595086506)
Action: inspect
Reward: 50.0
-> Observation: found
==== Step 9 ====
True state: shelf2
Belief: (ObjectState(shelf1): 0.023434919412759752, ObjectState(shelf2): 0.9290990911351868, ObjectState(shelf3): 0.023508414510322217, ObjectState(shelf4): 0.023957574941731195)
Action: inspect
Reward: 50.0
-> Observation: found
```

Figure 7: Shelf Robot 10-step Simulation (Object in Shelf 2)